π-Former: Succinct Proofs of Transformer Inference

Hideaki Takahashi

April 2026

Overview

Why transformers? The backbone of essentially every state-of-the-art language, vision, and speech model today (GPT, Claude, Llama, ViT, Whisper). As inference moves to remote APIs, users need cryptographic evidence that the *claimed* model produced the *claimed* output.

What is a SNARK? A *succinct non-interactive argument of knowledge*: a short proof that a specific computation was carried out correctly, verifiable in time far less than re-running it.

Problem. Build a SNARK for transformer inference $\mathbf{y} = \mathcal{M}_\theta(\mathbf{x})$ that the prover can actually run at scale.

Our approach: co-design. Modify the model so that its computation is natively sumcheck + Lasso friendly, then prove the new model *exactly* – rather than approximating a standard transformer.

Headline. 5.26M-param transformer, 12 CPU threads: **6.4 s prove / 0.37 s verify** – ~**8**× **faster prover** than zkGPT, transparent setup.

What Is a Transformer? (Intuition)

A transformer is a stack of L identical **blocks**. Each block takes a sequence of T token vectors $\mathbf{X} \in \mathbb{R}^{T \times d}$ and returns the same shape.

Inside one block, two operations:

1. **Attention**: each token looks at every other token and pulls in information from the relevant ones.
2. **Feed-forward (FFN)**: each token is independently passed through a small two-layer neural network.

Each is wrapped with: **LayerNorm** (rescale the row), a **residual** "+" (add the input back).

$$\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \mathsf{Att}(\mathsf{LN}(\mathbf{X}_{\text{in}})) \qquad \mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \text{FFN}(\mathsf{LN}(\mathbf{X}_{\text{mid}}))$$

That's it. Repeat L times, project to vocabulary, predict next token.

What Is Attention? (Intuition)

Each token is given three vectors:

- **Query $\mathbf{q}$** – "what am I looking for?"
- **Key $\mathbf{k}$** – "what do I offer?"
- **Value $\mathbf{v}$** – "what info do I carry?"

all linear projections of the input row: $\mathbf{q} = \mathbf{x}\mathbf{W}_Q$, etc.

For token t, attention computes a *weighted average of values*, where the weight is how well its query matches each key:

$$\mathbf{out}_t = \sum_s \underbrace{\mathbf{softmax}_s\big(\langle \mathbf{q}_t, \mathbf{k}_s \rangle / \sqrt{d}\big)}_{\text{attention weight}} \mathbf{v}_s .$$

What this gives the model. Tokens can "ask" for information from other positions and aggregate it – no fixed window, no recurrence.

What this costs the SNARK. Softmax means $\exp(\cdot)$ and row-wise division – both *very* expensive in a finite field. This is what we will replace.

The Other Two: GeLU and LayerNorm

GeLU. A smooth nonlinearity used inside the FFN, applied element-wise:

$$\mathrm{GeLU}(x) \;=\; x \cdot \Phi(x), \qquad \Phi = \text{Gaussian CDF}.$$

Like a soft ReLU. Lets the FFN learn nonlinear features.
SNARK pain: transcendental (CDF / exp).

LayerNorm. Stabilises training by rescaling each row to zero-mean / unit-variance, then re-scaling with learned γ, β:

$$\mathsf{LN}(\mathbf{x}) \;=\; \gamma \odot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sqrt{\sigma^2(\mathbf{x}) + \epsilon}} + \beta.$$

Without it, deep stacks blow up.
SNARK pain: a square root and a division per row.

So a transformer block has *four* hostile pieces

softmax, GeLU, LayerNorm, dense $\mathbf{W}$ – each adds either non-arithmetic operations or a field multiplication per weight.

Two Approaches: Prove-As-Is vs. Co-Design

Goal. Prove $\mathbf{y} = \mathcal{M}_\theta(\mathbf{x})$ with a short proof and cheap verification.

Prior work (zkLLM, zkGPT). Take a *standard* transformer and prove its computation *exactly*. Softmax, GeLU, LayerNorm are SNARK-hostile, so they are approximated with large lookup tables and fused constraints. The proven model is the original, but the prover pays minutes per token at GPT scale.

π-Former: co-design model and SNARK. Change the model itself so its computation is natively SNARK-friendly, and the SNARK proves the *new* model exactly:

Standard op	$\Rightarrow$	**π-Former op**
dense $\mathbf{W}$ (FP)	$\Rightarrow$	**ternary** $\mathbf{W} \in \{-1, 0, +1\} \cdot \alpha$ *(no field mults)*
softmax attention	$\Rightarrow$	**Leanable Lookup Activation**: linear attention with learnable $\boldsymbol{\phi}$
GeLU	$\Rightarrow$	same learnable additive sub-table lookup
LayerNorm	$\Rightarrow$	constraint-fused LN *(zkGPT-style)*

Toolbox (One-Line Reminders)

Multilinear extension (MLE). For a matrix $\mathbf{X} \in \mathbb{F}^{T \times d}$, its MLE $\widetilde{\mathbf{X}} : \mathbb{F}^{\log T + \log d} \to \mathbb{F}$ is the *unique* multilinear polynomial with $\widetilde{\mathbf{X}}(\mathbf{r}_t, \mathbf{r}_d) = \mathbf{X}[\mathbf{r}_t, \mathbf{r}_d]$ for all boolean $(\mathbf{r}_t, \mathbf{r}_d) \in \{0,1\}^{\log T + \log d}$. We commit MLEs so the prover can answer evaluations at random field points.

Sumcheck. Reduces $H = \sum_{x \in \{0,1\}^n} f(x)$ to a single evaluation $f(r)$ at random r. Verifier cost $O(n\delta)$, soundness error $\leq n\delta/|\mathbb{F}|$.

Hyrax Polynomial Commitment Scheme (PCS). Transparent (no trusted setup) polynomial commitment based on Pedersen / DL on BN254 G1. $O(\sqrt{N})$ commitment size, $O(\sqrt{N})$ open. Importantly: **additively homomorphic**.

Lasso Lookup. A lookup argument that proves $v_j = T[\mathrm{idx}_j]$ for many queries against a table T, *cheap when T decomposes into small additive sub-tables*.

Novel Idea 1: Ternary Projections

All projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$, LM head) are constrained:

$$\mathbf{W} \in \{-1, 0, +1\}^{m \times n}, \qquad \mathbf{y} = \alpha \cdot (\mathbf{x}\mathbf{W}) + \mathbf{b}$$

with one learnable scalar α per layer (BitNet b1.58 style).

What changes inside the SNARK? Consider the standard sumcheck for a contraction $\mathbf{Y} = \mathbf{X}\mathbf{W}$ (here $\widetilde{\mathbf{X}}, \widetilde{\mathbf{W}}, \widetilde{\mathbf{Y}}$ are the MLEs):

$$\widetilde{\mathbf{Y}}(r_t, r_d) - \mathbf{b}(r_d) \;=\; \alpha \sum_k \widetilde{\mathbf{X}}(r_t, k)\, \widetilde{\mathbf{W}}(k, r_d).$$

Because $\widetilde{\mathbf{W}}(k, r_d) \in \{-1, 0, +1\}$ after evaluation, each prover round is:

"$+= / -= /$ skip" – no field multiplications.

Novel Idea 2: Learnable Lookup Attention (LLA)

Linear attention replaces $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})$ by a positive-definite kernel $\kappa(\mathbf{q},\mathbf{k}) = \langle \boldsymbol{\phi}(\mathbf{q}), \boldsymbol{\phi}(\mathbf{k})\rangle$:

$$\mathrm{LLA}(\mathbf{Q},\mathbf{K},\mathbf{V})_t \;=\; \boldsymbol{\phi}(\mathbf{q}_t)^\top \underbrace{\sum_s \boldsymbol{\phi}(\mathbf{k}_s)\mathbf{v}_s^\top}_{=\mathbf{M}\ \text{(context, computed once)}} .$$

Cost: $O(n\,d^2)$ instead of $O(n^2 d)$, no exp, no row division.

Where's $\boldsymbol{\phi}$? We make it learnable and SNARK-friendly:

$$\boldsymbol{\phi}(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\mathrm{int}})\big],$$

i.e. split a b-bit input into c chunks, look each up in a small table T_i of size $2^{b/c}$, sum.

Why this matters.

- For b=16, c=2: two 256-entry tables instead of one 65,536-entry table.
- Init so $\sum_i T_i \approx$ GeLU, then train end-to-end with the straight-through estimator.

Re-using zkGPT-style Constraint-Fused LayerNorm

We adopt **zkGPT**'s constraint-fused LayerNorm.

What it does. Rewrite the LN identity as integer relations, prove them at *one* random row index r_t in four steps:

1. **Mean**: degree-2 sumcheck.
2. **Variance**: cubic sumcheck on the centered residual.
3. **Sigma range**: prove $\sigma = \lfloor \sqrt{\mathrm{var}}/d \rfloor$ via two range checks (no $\sqrt{}$ in the circuit).
4. **Y fusion**: one multi-batched cubic sumcheck for the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs together.

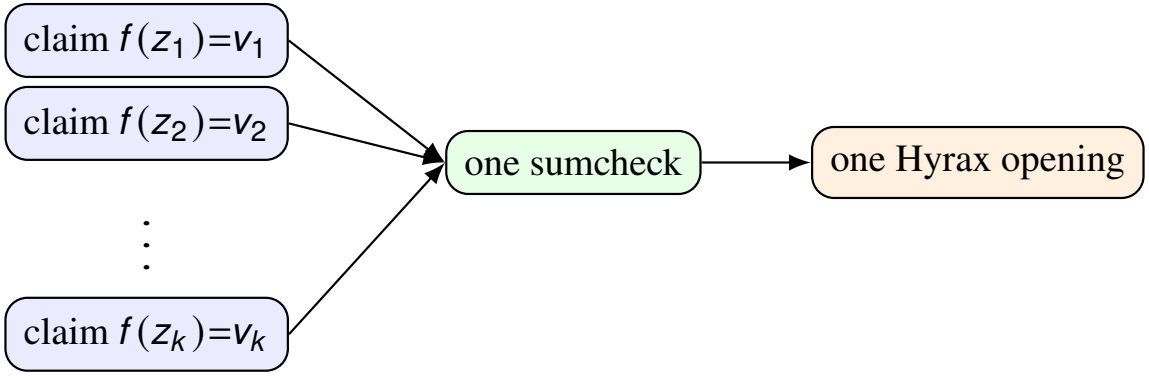
Result. No field divisions, no square roots. Verifier reads γ, β from the VK; everything else is a Hyrax opening.

Our novel pieces are LLA + ternary projections

LayerNorm is the SNARK-friendly piece we *re-use* from prior work.

Optimization: GKR and Batching

GKR-style combine. The same committed witness f (e.g. $\mathbf{X}_{n1}$) is evaluated by many sub-protocols at different points.



k claims on the same f $\longrightarrow$ **1 sumcheck + 1 opening**.

Cross-block batching. Same protocol type runs once per block. Fold the L per-block claims with Fiat–Shamir weights $\eta^{\ell-1}$ before the inner sumcheck.

Where we apply it.

- All Lasso instances $\rightarrow$ one global multi-Lasso.
- All LayerNorm range proofs $\rightarrow$ one shared multiplicity.
- Residuals $\rightarrow$ *no extra proof*; verifier just adds Hyrax commitments.

End-to-End Performance vs. zkGPT

x86_64 CPU, 12 threads. Times are pure compute; IO excluded.

		π-Former		zkGPT	
Model	**Params**	Prove	Verify	Prove	Verify
medium	924k	**2.92 s**	0.21 s	19.99 s	0.13 s
large	5.26M	**6.39 s**	0.37 s	52.59 s	0.44 s

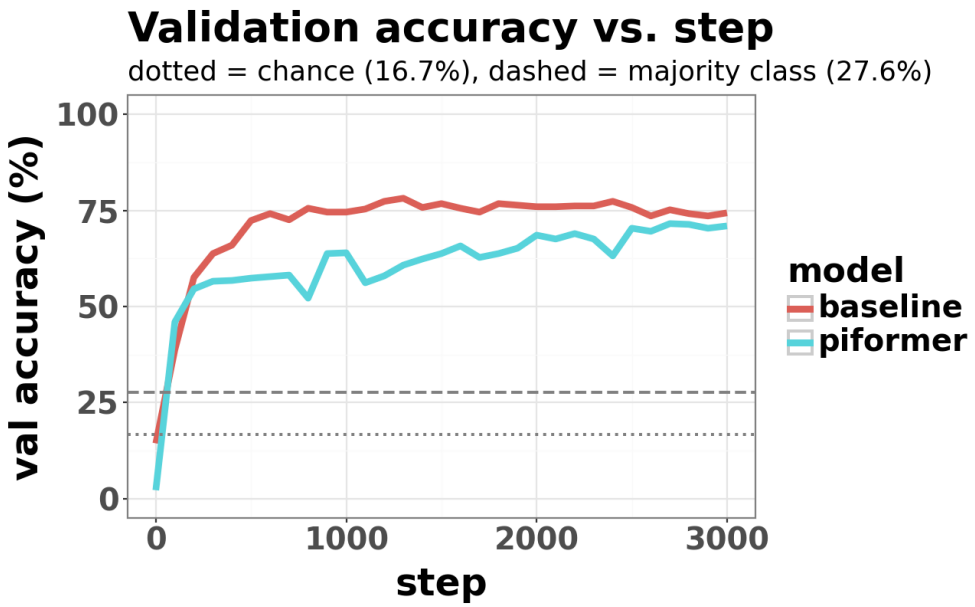
medium: d=128, L=4, T=32; large: d=256, L=6, T=32.

Headline.

- **Prover**: π-Former is ~**6.8**× faster at medium and ~**8.2**× faster at large.
- **Verifier**: comparable – zkGPT slightly faster at medium (0.13 s vs 0.21 s), π-Former faster at large (0.37 s vs 0.44 s).
- Both are transparent (no trusted setup).

Accuracy Sanity Check: TREC Question Classification

Task. TREC-6 *coarse* question type classification (Li & Roth, 2002): 6 classes (`ABBR` / `DESC` / `ENTY` / `HUM` / `LOC` / `NUM`). Char-level encoding, T=128, mean-pooled non-causal classifier on top of the encoder. Both models trained for 3,000 steps with the same optimizer.



Model	Val. acc.
Baseline	74.4%
π-Former	71.0%

~82k params each, 3,000 steps.

Take-aways

1. **Co-design beats approximation.** Replace softmax / GeLU / LN / dense $\mathbf{W}$ with twins that are *natively* sumcheck + Lasso.
2. **Our novel pieces**: LLA (linear, lookup-based attention) and ternary projections (no in-circuit field mults). LN re-uses zkGPT-style fusion.
3. **Cross-block batching** folds L per-block proofs into 6 model-level sumchecks, 1 global Lasso, 1 global range, 10 deferred openings.
4. **Result.** 6-layer, 5.26M-parameter transformer: **6.4 s prove / 0.37 s verify** ($\sim 8\times$ faster prover than zkGPT) on 12 CPU threads, transparent setup.

Thank you.

ht2673@columbia.edu